The Transformation of Optimal Independent Spanning Trees in Hypercubes*

Shyue-Ming Tang[1]†, Yu-Ting Li[2], and Yue-Li Wang[2]

[1]Fu Hsing Kang School,
National Defense University, Taipei, Taiwan, R.O.C.
[2]Department of Information Management,
National Taiwan University of Science and Technology, Taipei, Taiwan, R.O.C.

Abstract

Multiple spanning trees rooted at the same vertex, say r, of a given graph are said to be independent if for each non-root vertex, say v, paths from r to v, one path in each spanning trees, are internally disjoint. It has been proved that there exist k optimal independent spanning trees (OIST for short) rooted at any vertex in the k-dimensional hypercube. The word optimal is defined by an additional requirement: in each spanning tree, the distance from v to the only child of r must be the Hamming distance. In this paper, we shall propose an algorithm to generate $(k-1)!$ instances of OIST according to an existing OIST (including itself).

Keywords: optimal independent spanning trees, hypercubes, internally disjoint paths, Hamming distance, derangement number, fault-tolerant broadcasting.

1. Introduction

Two spanning trees of G are called *independent* if they are rooted at the same vertex, say r, and for each vertex $v \in V \setminus \{r\}$, the two paths from r to v, one path in each tree, are internally disjoint. A set of spanning trees on a graph is said to be independent if they are pairwise independent [7]. In 1989, Zehavi and Itai conjecture that, for any vertex r in a k-connected graph G, there exist k independent spanning trees (IST for short) rooted at r in G [13]. Although the conjecture has been proved for k-connected graphs with $k \leqslant 4$ [1,3,5], it is still open for $k > 4$.

*This work was supported by the National Science Council, Republic of China, under Contract NSC 100–2410–H–606–007.

†All correspondence should be addressed to Shyue-Ming Tang, Fu Hsing Kang School, National Defense University, 70, Section 2, Zhong-Yang N. Road, Taipei, Taiwan 11258, Republic of China (e-mail: stang@ndu.edu.tw).

Broadcasting in a computer network becomes fault-tolerant when it is equipped with a protocol based on IST [5]. Message can be disseminated from a source vertex to all other vertices by sending k copies of the message along k IST rooted at the source vertex. If the source vertex is faultless, this scheme can tolerate up to $k-1$ faulty vertices.

A k-dimensional hypercube, also called k-cube, can be represented by a graph $G = (V, E)$ with $V = \{0, 1, 2, \ldots, 2^k - 1\}$ and $E = \{(u, v) | u \oplus v = 2^i, 0 \leqslant i \leqslant k-1\}$, where $\oplus$ denotes a k-bit exclusive or operation. Hypercubes form a subclass of bipartite graphs, as well as a subclass of product graphs [4,9].

Hypercubes are a very famous topology for developing network algorithms. Some of the studies are concerned with fault-tolerant broadcasting [2, 6, 8, 10]. In [11], Tang et al. first from the viewpoint of IST, designed an algorithm to solve the fault-tolerant broadcasting problem on hypercubes. They also first proposed the concept of *optimal* independent spanning trees (OIST for short) on hypercubes. A parallel version of this algorithm was proposed in [12]. The parallel algorithm relies on a simple rule and can be implemented easily. It has been proved in [11] that there exist k OIST rooted at any vertex in the k-cube. The word *optimal* is defined by adding a requirement to each spanning tree, that is, the distance from a non-root vertex to the only child of the root must be the Hamming distance. Following the definition of OIST, we use an OIST to substitute an instance of k OIST hereinafter for concise sake. An OIST on the 4-cube are shown in Figure 1.

The main purpose of this paper is to propose an algorithm to transform an existing OIST on the k-cube to another OIST. This transformation can be executed consecutively and generate $(k-1)!$ OISTs according to the existing OIST (including itself). The point is that more OISTs are generated, more options can be provided when a fault-tolerant broadcasting system is built on a hypercube.

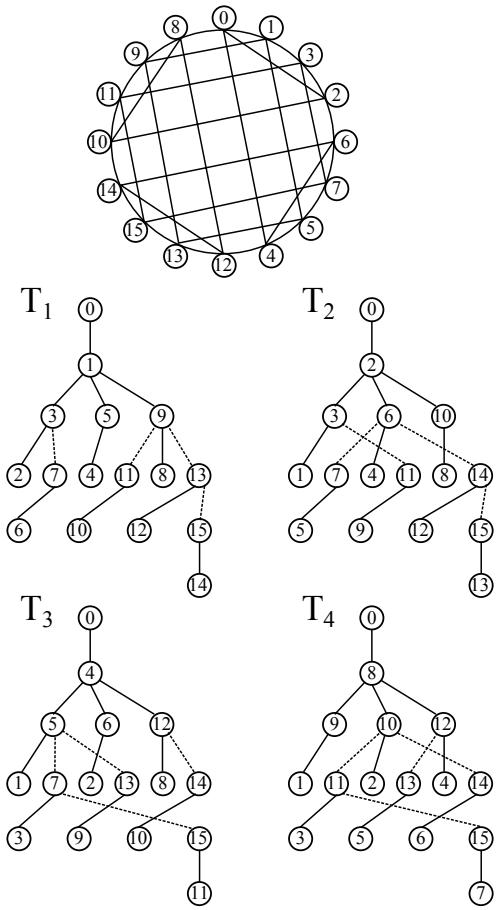

Figure 1: The 4-cube and an OIST on it.

The remaining part of this paper is organized as follows. In Section 2, we introduce some important properties of OIST on hypercubes. Section 3 contains our generation algorithm and a correctness proof of the algorithm. The last section gives our concluding remarks.

2. Important Properties of OIST on Hypercubes

In this section, we propose some fundamental properties of OIST on hypercubes. Since hypercubes are vertex-symmetric, without loss of generality, we can only consider vertex 0 as the root vertex of all the tree. Further, we use $x_k x_{k-1} \dots x_1$ as the binary representation of vertex x in the k-cube, where x_i= 0 or 1 is the bit at position i, for $1 \leqslant i \leqslant k$.

Due to the internally disjoint requirement, it is obvious that the root has only one child in every spanning tree. Suppose $x = 2^{j-1}$ is the only child of the root, i.e., $x_j = 1$ and $x_i = 0$ for $i \neq j$. The spanning tree is denoted by T_j. In Figure 1, each of the spanning trees is named with the only child of the root.

According to the definition of hypercubes, any two vertices are adjacent if and only if their binary representations have one different bit. We define *parent*(x, j) as the position of the different bit of vertex x with its parent in tree T_j. That is, if parent$(x, j) = i$ and vertex y is the parent of x in T_j, then either $y = x-2^i$ or $y = x+2^i$ when $x_i = 1$ or 0, respectively. We introduce the following two important lemmas which form necessary conditions for constructing an OIST on hypercubes.

Lemma 1. *In an OIST on the k-cube, if* $x_i = 1$ *and* x *is not the child of the root, then parent(x,i)* $\neq i$ *for* $1 \leqslant i \leqslant k$.

Proof. Let y be the parent of x and z be the child of the root. If $x_i = 1$ and parent$(x,i) = i$, then y_i = 0. Since $z_i = 1$, the distance from x to z in T_i must be greater than the Hamming distance of x and z. It is a contradiction to the OIST definition. □

Lemma 2. *In an OIST on the k-cube, if* $x_i = 0$, *then parent(x,i)* $= i$ *for* $1 \leqslant i \leqslant k$.

Proof. Suppose parent$(x,i) \neq i$. There must be a T_j such that parent$(x,j) = i$ and $i \neq j$. Let y be the parent of x and z be the child of the root in T_j. If $x_i = 0$ and parent$(x,j) = i$, then $y_i = 1$. Since z_i = 0 ($z_j = 1$), the distance from x to z in T_j must be greater than the Hamming distance of x and z. It is a contradiction to the OIST definition. □

Corollary 3. *In an OIST on the k-cube, if* $x_i = 0$, *then* x *is a leaf in* T_i.

Proof. Suppose x has a child y in T_i. Since x_i = 0, x is not the root child of T_i. If $y_i = 1$, then parent$(y, i) \neq i$ (by Lemma 1), i.e., $x_i = 1$. If $y_i = 0$, then parent$(y, i) = i$ (by Lemma 2), i.e., $x_i = 1$. Both cases result in a contradiction to $x_i = 0$. Therefore, x has no child. □

Corollary 4. *In an OIST on the k-cube, there are* $2^{k-1} - 1$ *leaves in every tree.*

Proof. By Corollary 3, vertex x is a leaf of T_i if $x_i = 0$. The number of 1-bits in x may vary from 1 to $k - 1$ and there are $k - 1$ positions are available because the i-th bit is reserved for the 0-bit. Thus, the number of leaves is $\sum_{i=1}^{k-1} C_i^{k-1} = 2^{k-1} - 1$. □

For the root, its only child in every tree is distinct, while for a non-root vertex, its parent in every

tree is also distinct. It turns out that the IST construction problem is to determine a bijection from the parent set to the root-child set for every non-root vertex. By Lemma 2, the parent of vertex x in T_i is determined when $x_i = 0$. We have to know how many vertices x can choose as its parent in T_i when $x_i = 1$. And then, we can determine how many possible parent combination for x in an OIST.

The starting point is an integer sequence, called *subfactorial* or *derangement number*, which is defined as the number of permutations of n elements with none at its original position. Let $!n$ denote the derangement number of n elements. We have !1=0, !2=1, !3=2, !4=9, !5=44, and so forth. Euler has proved both recurrences $!n = (n-1)(!(n-1)+!(n-2))$ and $!n = n\cdot!(n-1) + (-1)^n$ [14].

Based on Lemmas 1 and 2, we have the following theroy and corollaries.

Theorem 5. *For a vertex x in the k-cube, $x \neq 0$ and $x \neq 2^i (i = 0, 1, \ldots, k-1)$, if x has m number of 1-bits ($m \geqslant 2$), then the number of possible combinations of its parents in each tree of an OIST is $!m$.*

Proof. Let $1 \leqslant i, j \leqslant k$. Based on Lemma 2, parent(x,i)= i if $x_i = 0$. The parent of x in tree T_i is fixed for $x_i = 0$. Based on Lemma 1, on the other hand, parent(x,i)= j if $x_i = 1$, where $i \neq j$. Since $m \geqslant 2$, there must exist a j such that $x_j = 1$. That is, vertex x reaches its parent in T_i by changing x_j from 1 to 0. Therefore, the number of different parent combinations in those trees is the derangement number of m. □

Corollary 6. *For a vertex x in the k-cube, $x \neq 0$, if the number of 1-bits in x is less than three, then its parent in each tree of an OIST is fixed.*

Proof. If x has one 1-bit, say x_j= 1, then x is the only child of the root in T_j. For $i \neq j$, parent(x,i)= i since x_i= 0 (Lemma 2). In case that x has two 1-bits, by Theorem 5, its parent combination in each tree has only one choice. □

If the number of 1-bits in x is great than two, its parent in each tree of an OIST has more than one choice. We call it a *parent-unfixed vertex*. We can also infer that the number of parent-unfixed vertices in an OIST is $\sum_{i=3}^{k} C_i^k = 2^k - k - 1$.

In Figure 1, the solid lines in the OIST represent the fixed relationship with parents, while the dot lines represent the unfixed relationship. In the 4-cube, vertices 1(0001), 2(0010), 3(0011), 4(0100), 5(0101), 6(0110), 8(1000), 9(1001), 10(1010) and 12(1100) have fixed parents relationship (by Corollary 6). Vertices 7(0111), 11(1011), 13(1101) and 14(1110) have !3=2 choices of parent. As for vertex 15(1111), !4=9 choices are available. Therefore, we can use a brute force approach to find 60 OISTs out of $2^4 \cdot 9^1 = 144$ parent combinations.

In the 5-cube, the number of parent combinations greatly increases to $2^{10} \cdot 9^5 \cdot 44^1 = 2,660,511,744$. We found out 13,977,216 OISTs by running a program. From such huge amount of solutions, we can understand why there were so many algorithms published for the fault-tolerant broadcasting problem in hypercubes.

3. A Transformation from an Existing OIST

In this section, we present an algorithm for generating $(k-1)!$ OISTs according to an existing OIST on the k-cube.

Based on Lemma 2 and Corollary 3, all leaves in an OIST have their fixed parents. According to Corollary 6, all vertices with one or two 1-bits are also have their fixed parents in an OIST. Together with Corollary 4, we infer that there are $2^{k-1} - k$ internal vertices in a tree have more than one parent choices. (Note that the number of parent-unfixed vertices in an OIST is greater than it.)

Suppose that vertex x has m 1-bits. We can infer that x is an internal vertex in m trees, and in each tree x has a shortest path (with Hamming distance) to the root. Since the number of 1-bits must be decreasing vertex by vertex, an internal vertex and its ancestors are decreasingly ordered in a tree.

Suppose an internal vertex x takes *jump* j to reach its parent y in a tree. That means $y = x - 2^{j-1}$. A *jump array* of vertex x, denoted by $L_x(i, j)$, is an $m \times m$ array where stores the j-th jump from x to the root in tree T_i. Note that we always keep $L_x(i, m) = i$ in a jump array. Then, the i-th row of L_x stores a shortest path from x to the root of T_i for $x_i = 1$. To meet the requirement of internally disjoint paths, the first p jumps in each row must be different from those in any other row for $p < m$.

A procedure to obtain the jump array of a parent-unfixed vertex is depicted as follows.

Procedure GET_JUMP_ARRAY
input: a non-zero vertex x and an OIST on the k-cube, i.e., parent(i,j) for $1 \leqslant i \leqslant 2^{k-1} - 1$ and $1 \leqslant j \leqslant k$.
output: the jump array of x, L_x.
begin
m = the number of 1-bits in x;

for $i = 1$ to k **do**
$current\_v = x$;
if $x_i \neq 0$ **then**
for $i = 1$ to m **do**
$L_x(i, j) =$ parent(x,i);
$current\_v = current\_v - 2^{L_x(i,j)}$;
end do
end If
end do
end of GET_JUMP_ARRAY

Procedure GET_JUMP_ARRAY is $O(m^2)$ in both time and space complexity where m is the number of 1-bits in x. Taking the OIST in Figure 1 as an example, the jump arrays of parent-unfixed vertices 7, 11, 13, 14 and 15 are:

$$L_7 = \begin{bmatrix} 3 & 2 & 1 \\ 1 & 3 & 2 \\ 2 & 1 & 3 \end{bmatrix}, \quad L_{11} = \begin{bmatrix} 2 & 4 & 1 \\ 4 & 1 & 2 \\ 1 & 2 & 4 \end{bmatrix},$$

$$L_{13} = \begin{bmatrix} 3 & 4 & 1 \\ 4 & 1 & 3 \\ 1 & 3 & 4 \end{bmatrix}, \quad L_{14} = \begin{bmatrix} 4 & 3 & 2 \\ 2 & 4 & 3 \\ 3 & 2 & 4 \end{bmatrix}, \text{ and}$$

$$L_{15} = \begin{bmatrix} 2 & 3 & 4 & 1 \\ 1 & 4 & 3 & 2 \\ 4 & 2 & 1 & 3 \\ 3 & 1 & 2 & 4 \end{bmatrix}.$$

The basic idea for transforming an OIST is to consistently change the jump order of every path to the root. This change is completed by swapping two jumps in every jump array that contains at least one of the two jumps. When a jump array of a parent-unfixed vertex x contains the two jumps, we swap the two jumps in every row, and the jump order of every path from x to the root is changed consistently.

Two vertices are *spouses* if they have a common child in two trees of an OIST. Let y be a *spouse vertex* of x. Vertices x and y must have the same number of 1-bits, and the Hamming distance between x and y is two. In some cases, a jump array might contain only one of the swapped jumps. We deal with this case by changing the jump to another one. This change causes a swap of two jump arrays which belong to spouses. If a jump array contains none of the swapped jumps, we ignore the vertex.

We swap two jumps in each transformation. The following Algorithm describes the transformation.

Algorithm TRANSFORM_OIST
input: an OIST on the k-cube and two swapping jumps (a,b) where $1 \leqslant a, b \leqslant k$.
output: a new OIST.
begin
Step 1. Obtain jump arrays for all parent-unfixed vertices.
call Procedure GET_JUMP_ARRAY;
Step 2. Swap jumps a and b in all jump arrays.
for each parent-unfixed vertex x **do**
Step 2.1.
change all a to b and all b to a if they both exist in L_x;
Step 2.2.
change all a to b if only a exist in L_x;
(L_x becomes L_y where y is the spouse vertex of x with respect to (a, b))
Step 2.3.
change all b to a if only b exist in L_x;
(L_x becomes L_y where y is the spouse vertex of x with respect to (a, b))
end do
Step 3. Obtain a new OIST from all jump arrays.
for every parent-unfixed vertex x **do**
$m =$ the number of 1-bits in x;
for $j = 1$ to m **do**
$i = L_x(j, m)$;
parent(x,i) $= L_x(i, 1)$;
end do
end do
end of TRANSFORM_OIST

The time complexity of Procedure TRANSFORM_OIST is $O(n \cdot log^2 n)$ where $n = 2^k$ is the number of vertices in the k-cube. For example, we swap jumps 3 and 4 in the jump arrays above, and transform to the following jump arrays for each parent-unfixed vertex in the 4-cube:

$$L_7^* = \begin{bmatrix} 2 & 3 & 1 \\ 3 & 1 & 2 \\ 1 & 2 & 3 \end{bmatrix}, \quad L_{11}^* = \begin{bmatrix} 4 & 2 & 1 \\ 1 & 4 & 2 \\ 2 & 1 & 4 \end{bmatrix},$$

$$L_{13}^* = \begin{bmatrix} 4 & 3 & 1 \\ 1 & 4 & 3 \\ 3 & 1 & 4 \end{bmatrix}, \quad L_{14}^* = \begin{bmatrix} 3 & 4 & 2 \\ 4 & 2 & 3 \\ 2 & 3 & 4 \end{bmatrix}, \text{ and}$$

$$L_{15}^* = \begin{bmatrix} 2 & 4 & 3 & 1 \\ 1 & 3 & 4 & 2 \\ 4 & 1 & 2 & 3 \\ 3 & 2 & 1 & 4 \end{bmatrix}.$$

By Step 2.1 of Algorithm TRANSFORM_OIST, L_{15}^*, L_{13}^* and L_{14}^* are obtained by swapping jumps 3 and 4 in L_{15}, L_{13} and L_{14}, respectively. By Step 2.2, L_{11}^* is obtained by changing jump 3 to jump 4 in L_7. By Step 2.3, L_7^* is obtained by changing jump 4 to jump 3 in L_{11}. It turns out that vertices 7 and 11 are spouses with respect to jumps 3 and 4. The transformed OIST are shown in Figure 2.

To show the correctness of Algorithm TRANSFORM_OIST, we have to prove that the transformed result is an OIST.

Theorem 7. *The output graph of Algorithm* TRANSFORM_OIST *is an OIST.*

Proof. We prove the theorem in a concise manner. Firstly, the spanning tree property is kept in

the new jump arrays. Secondly, the property of internally disjoint paths is also kept in the new jump arrays. □

Obviously, different input jump pairs of Algorithm TRANSFORM_OIST result in different OISTs. If we want to generate $(k-1)!$ OISTs from an OIST on the k-cube, we can run Algorithm TRANSFORM_OIST with adequate input jump pairs. The transformation continues until all jump orders have been generated. For example, the first row of L_{15} in the example is (2,3,4,1). By fixing the last jump, five continuous jump swaps on the jump arrays make the row become (2,4,3,1), (4,2,3,1), (4,3,2,1), (3,4,2,1) and (3,2,4,1). Five different OISTs are generated accordingly.

4. Concluding Remarks

In this paper, we study the properties of OISTs on hypercubes and propose an algorithm to generate $(k-1)!$ OISTs according to an existing OIST on the k-cube. This algorithm swaps jumps pairwise in all jump array, and keeps the internally disjoint property of paths from a vertex to the root in different spanning trees.

We are also concerned with the minimum transformation between different OISTs, and the construction of OISTs under the limitation of some fixed paths. We shall expand our study to these issues in the near future.

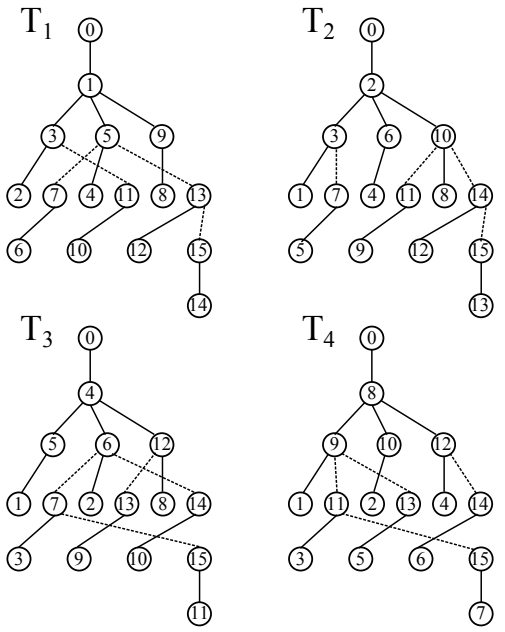

Figure 2: A transformed OIST on the 4-cube.

References

[1] J. Cheriyan and S. N. Maheshwari, Finding Nonseparating Induced Cycles and Independent Spanning Trees in 3-Connected Graphs, *Journal of Algorithms* 9 (1988), pp.507–537.

[2] G.-M. Chiu, A Fault-tolerant Broadcasting Algorithm for Hypercubes, *Information Processing Letters*, 66 (1998), pp.93–99.

[3] S. Curran, O. Lee, and X. Yu, Finding four independent trees, *SIAM Journal on Computing* 35 (2006), pp.1023–1058.

[4] W. D. Hillis and L. W. Tucker, The CM-5 Connection Machine: A Scalable Supercomputer, *Communications of the ACM* 36 (1993), pp.30–40.

[5] A. Itai and M. Rodeh, The Multi-Tree Approach to Reliability in Distributed Networks, *Proceedings of the 25th Annual IEEE Symposium on Fundamental Computer Science*, 1984, pp.137–147. (also in *Information and Computation* 79 (1988), pp.43–59.)

[6] S. L. Johnsson and C. T. Ho, Optimum Broadcasting and Personalized Communication in Hypercubes, *IEEE Transactions on Computers* 38 (1989), pp.1249–1268.

[7] S. Khuller and B. Schieber, On Independent Spanning Trees, *Information Processing Letters* 42 (1992), pp.321–323.

[8] P. Ramanathan and K.G. Shin, Reliable broadcast in hypercube multicomputers, *IEEE Transactions on Computers* 37 (1988), pp.1654–1657.

[9] Y. Saad and M. H. Schultz, Topological Properties of Hypercube, *IEEE Transactions on Computers* 37 (1988), No. 7, pp.867–872.

[10] T.-Y. Sung, M.-Y. Lin, and T.-Y. Ho, Multiple-edge-fault Tolerance with respect to Hypercubes, *IEEE Transactions on Parallel and Distributed Systems* 8 (1997), pp.187–192.

[11] S.-M. Tang, Y.-L. Wang, Y.-H. Leu, Optimal Independent Spanning Trees on Hypercubes, *Journal of Information Science and Engineering* 20 (2004), pp.143–155.

[12] J.-S. Yang, S.-M. Tang, J.-M. Chang and Y.-L. Wang, Parallel Construction of Optimal Independent Spanning Trees on Hypercubes, *Parallel Computing*, 33 (2007), pp.73–79.

[13] A. Zehavi and Alon Itai, Three Tree-paths, *Journal of Graph Theory* 13 (1989), pp.175–188.

[14] J. C. Baez, Let's get deranged ! (2003), refer to http://math.ucr.edu/home/baez/qg-winter2004/derangement.pdf, also refer to http://en.wikipedia.org/wiki/Derangement.